

custom information, such as an email address or login credentials. When your app creates a **QuickLink**, the app returns the **QuickLink** to the system by calling [ReportCompleted](#).

A **QuickLink** does not actually store data. Instead, it contains an identifier that, when selected, is sent to your app. Your app is responsible for storing the Id of the **QuickLink** and the corresponding user data. When the user taps the **QuickLink**, you can get its Id through the [QuickLinkId](#) property.

C#

```
async void ReportCompleted(ShareOperation shareOperation, string
quickLinkId, string quickLinkTitle)
{
    QuickLink quickLinkInfo = new QuickLink
    {
        Id = quickLinkId,
        Title = quickLinkTitle,

        // For quicklinks, the supported FileTypes and DataFormats are set
        // independently from the manifest
        SupportedFileTypes = { "*" },
        SupportedDataFormats = { StandardDataFormats.Text,
StandardDataFormats.Uri,
                                StandardDataFormats.Bitmap, StandardDataFormats.StorageItems
        }
    };

    StorageFile iconFile = await
Windows.ApplicationModel.Package.Current.InstalledLocation.CreateFileAsync(
    "assets\\user.png", CreationCollisionOption.OpenIfExists);
    quickLinkInfo.Thumbnail =
RandomAccessStreamReference.CreateFromFile(iconFile);
    shareOperation.ReportCompleted(quickLinkInfo);
}
```

See also

- [App-to-app communication](#)
- [Share data](#)
- [OnShareTargetActivated](#)
- [ReportStarted](#)
- [ReportError](#)
- [ReportCompleted](#)
- [ReportDataRetrieved](#)
- [ReportStarted](#)

- [QuickLink](#)
- [QuickLinkId](#)

Copy and paste

Article • 10/20/2022

This article explains how to support copy and paste in Universal Windows Platform (UWP) apps using the clipboard. Copy and paste is the classic way to exchange data either between apps, or within an app, and almost every app can support clipboard operations to some degree. For complete code examples that demonstrate several different copy and paste scenarios, see the [clipboard sample](#).

Check for built-in clipboard support

In many cases, you do not need to write code to support clipboard operations. Many of the default XAML controls you can use to create apps already support clipboard operations.

Get set up

First, include the [Windows.ApplicationModel.DataTransfer](#) namespace in your app. Then, add an instance of the [DataPackage](#) object. This object contains both the data the user wants to copy and any properties (such as a description) that you want to include.

C#

```
DataPackage dataPackage = new DataPackage();
```

Copy and cut

Copy and cut (also referred to as *move*) work almost exactly the same. Choose which operation you want by using the [RequestedOperation](#) property.

C#

```
// copy  
dataPackage.RequestedOperation = DataPackageOperation.Copy;  
// or cut  
dataPackage.RequestedOperation = DataPackageOperation.Move;
```

Set the copied content

Next, you can add the data that a user has selected to the [DataPackage](#) object. If this data is supported by the [DataPackage](#) class, you can use one of the corresponding [methods](#) of the [DataPackage](#) object. Here's how to add text by using the [SetText](#) method:

C#

```
dataPackage.SetText("Hello World!");
```

The last step is to add the [DataPackage](#) to the clipboard by calling the static [SetContent](#) method.

C#

```
Clipboard.SetContent(dataPackage);
```

Paste

To get the contents of the clipboard, call the static [GetContent](#) method. This method returns a [DataPackageView](#) that contains the content. This object is almost identical to a [DataPackage](#) object, except that its contents are read-only. With that object, you can use either the [AvailableFormats](#) or the [Contains](#) method to identify what formats are available. Then, you can call the corresponding [DataPackageView](#) method to get the data.

C#

```
async void OutputClipboardText()
{
    DataPackageView dataPackageView = Clipboard.GetContent();
    if (dataPackageView.Contains(StandardDataFormats.Text))
    {
        string text = await dataPackageView.GetTextAsync();
        // To output the text from this example, you need a TextBlock
        control
        TextOutput.Text = "Clipboard now contains: " + text;
    }
}
```

Track changes to the clipboard

In addition to copy and paste commands, you may also want to track clipboard changes. Do this by handling the clipboard's [ContentChanged](#) event.

C#

```
Clipboard.ContentChanged += async (s, e) =>
{
    DataPackageView dataPackageView = Clipboard.GetContent();
    if (dataPackageView.Contains(StandardDataFormats.Text))
    {
        string text = await dataPackageView.GetTextAsync();
        // To output the text from this example, you need a TextBlock
control
        TextOutput.Text = "Clipboard now contains: " + text;
    }
}
```

See also

- [Clipboard sample](#) 
- [App-to-app communication](#)
- [DataTransfer](#)
- [DataPackage](#)
- [DataPackageView](#)
- [DataPackagePropertySet](#)
- [DataRequest](#)
- [DataRequested](#)
- [FailWithDisplayText](#)
- [ShowShareUi](#)
- [RequestedOperation](#)
- [ControlsList](#)
- [SetContent](#)
- [GetContent](#)
- [AvailableFormats](#)
- [Contains](#)
- [ContentChanged](#)

Interprocess communication (IPC)

Article • 11/01/2022

This topic explains various ways to perform interprocess communication (IPC) between Universal Windows Platform (UWP) applications and Win32 applications.

App services

App services enable applications to expose services that accept and return property bags of primitives ([ValueSet](#)) in the background. Rich objects can be passed if they're [serialized](#) [↗].

App services can run either [out of process](#) as a background task, or [in process](#) within the foreground application.

App services are best used for sharing small amounts of data where near real-time latency isn't required.

COM

[COM](#) is a distributed object-oriented system for creating binary software components that can interact and communicate. As a developer, you use COM to create reusable software components and automation layers for an application. COM components can be in process or out of process, and they can communicate via a [client and server](#) model. Out-of-process COM servers have long been used as a means for [inter-object communication](#).

Packaged applications with the [runFullTrust](#) capability can register out-of-process COM servers for IPC via the [package manifest](#). This is known as [Packaged COM](#) [↗].

Filesystem

BroadFileSystemAccess

Packaged applications can perform IPC using the broad filesystem by declaring the [broadFileSystemAccess](#) restricted capability. This capability grants [Windows.Storage](#) APIs and [xxxFromApp](#) Win32 APIs access to the broad filesystem.

By default, IPC via the filesystem for packaged applications is restricted to the other mechanisms described in this section.

PublisherCacheFolder

The [PublisherCacheFolder](#) enables packaged applications to declare folders in their manifest that can be shared with other packages by the same publisher.

The shared storage folder has the following requirements and restrictions:

- Data in the shared storage folder is not backed up or roamed.
- The user can clear the contents of the shared storage folder.
- You can't use the shared storage folder to share data among applications from different publishers.
- You can't use the shared storage folder to share data among different users.
- The shared storage folder doesn't have version management.

If you publish multiple applications and you're looking for a simple mechanism to share data between them, then the PublisherCacheFolder is a simple filesystem-based option.

SharedAccessStorageManager

[SharedAccessStorageManager](#) is used in conjunction with App services, protocol activations (for example, [LaunchUriForResultsAsync](#)), etc., to share [StorageFiles](#) via tokens.

FullTrustProcessLauncher

With the [runFullTrust](#) capability, packaged applications can [launch full trust processes](#) within the same package.

For scenarios where package restrictions are a burden, or IPC options are lacking, an application could use a full trust process as a proxy to interface with the system, and then IPC with the full trust process itself via App services or some other well supported IPC mechanism.

LaunchUriForResultsAsync

[LaunchUriForResultsAsync](#) is used for simple ([ValueSet](#)) data exchange with other packaged applications that implement the [ProtocolForResults](#) activation contract. Unlike App services, which typically run in the background, the target application is launched in the foreground.

Files can be shared by passing [SharedStorageAccessManager](#) tokens to the application via the [ValueSet](#).

Loopback

Loopback is the process of communicating with a network server listening on localhost (the loopback address).

To maintain security and network isolation, loopback connections for IPC are blocked by default for packaged applications. You can enable loopback connections among trusted packaged application using [capabilities](#) and [manifest properties](#).

- All packaged applications participating in loopback connections will need to declare the `privateNetworkClientServer` capability in their [package manifests](#).
- Two packaged applications can communicate via loopback by declaring [LoopbackAccessRules](#) within their package manifests.
 - Each application must list the other in its [LoopbackAccessRules](#). The client declares an "out" Rule for the server, and the server declares "in" Rules for its supported clients.

ⓘ Note

The package family name required to identify an application in these Rules can be found via the package manifest editor in Visual Studio during development time, via **Partner Center** for applications published through the Microsoft Store, or via the **Get-AppxPackage** PowerShell command for applications that are already installed.

Unpackaged applications and services don't have package identity, so they can't be declared in [LoopbackAccessRules](#). You can configure a packaged application to connect via loopback with unpackaged applications and services via [CheckNetIsolation.exe](#), however this is only possible for sideload or debugging scenarios where you have local access to the machine, and you have administrator privileges.

- All packaged applications participating in loopback connections need to declare the `privateNetworkClientServer` capability in their [package manifests](#).
- If a packaged application is connecting to an unpackaged application or service, run `CheckNetIsolation.exe LoopbackExempt -a -n=<PACKAGEFAMILYNAME>` to add a loopback exemption for the packaged application.
- If an unpackaged application or service is connecting to a packaged application, run `CheckNetIsolation.exe LoopbackExempt -is -n=<PACKAGEFAMILYNAME>` to enable the packaged application to receive inbound loopback connections.
 - [CheckNetIsolation.exe](#) must be running continuously while the packaged application is listening for connections.

- The `-is` flag was introduced in Windows 10, version 1607 (10.0; Build 14393).

⚠ Note

The package family name required for the `-n` flag of **CheckNetIsolation.exe** can be found via the package manifest editor in Visual Studio during development time, via **Partner Center** for applications published through the Microsoft Store, or via the **Get-AppxPackage** PowerShell command for applications that are already installed.

CheckNetIsolation.exe is also useful for [debugging network isolation issues](#).

Pipes

[Pipes](#) enable simple communication between a pipe server and one or more pipe clients.

[Anonymous pipes](#) and [named pipes](#) are supported with the following constraints:

- By default, named pipes in packaged applications are supported only between processes within the same package, unless a process is full trust.
- Named pipes can be shared across packages following the guidelines for [sharing named objects](#).
- Named pipes (in packaged *and* unpackaged apps) must use the syntax `\\.\pipe\LOCAL\` for the pipe name.

Registry

[Registry](#) usage for IPC is generally discouraged, but it is supported for existing code. Packaged applications can access only registry keys that they have permission to access.

Commonly, packaged desktop apps (see [Building an MSIX package from your code](#)) leverage [registry virtualization](#) such that global registry writes are contained to a private hive within the MSIX package. This enables source code compatibility while minimizing global registry impact, and can be used for IPC between processes in the same package. If you must use the registry, this model is preferred versus manipulating the global registry.

RPC

[RPC](#) can be used to connect a packaged application to a Win32 RPC endpoint, provided that the packaged application has the correct capabilities to match the ACLs on the RPC endpoint.

Custom capabilities enable OEMs and IHVs to [define arbitrary capabilities](#), [ACL their RPC endpoints with them](#), and then [grant those capabilities to authorized client applications](#). For a full sample application, see the [CustomCapability](#) [↗](#) sample.

RPC endpoints can also be ACLed to specific packaged applications to limit access to the endpoint to just those applications without requiring the management overhead of custom capabilities. You can use the [DeriveAppContainerSidFromAppContainerName](#) API to derive a SID from a package family name, and then ACL the RPC endpoint with the SID as shown in the [CustomCapability](#) [↗](#) sample.

Shared Memory

[File mapping](#) can be used to share a file or memory between two or more processes with the following constraints:

- By default, file mappings in packaged applications are supported only between processes within the same package, unless a process is full trust.
- File mappings can be shared across packages following the guidelines for [sharing named objects](#).

Shared memory is recommended for efficiently sharing and manipulating large amounts of data.

Networking and web services

Article • 10/20/2022

The following networking and web services technologies are available for Universal Windows Platform (UWP) developers.

Topic	Description
Networking basics	Things you must do for any network-enabled app.
Which networking technology?	An overview of the networking technologies available for a UWP developer, with suggestions on how to choose the technologies that are right for your app.
Network communications in the background	To continue network communication while it's not in the background, an app can use background tasks and either socket broker or control channel triggers.
Sockets	Sockets are a low-level data transfer technology on top of which many networking protocols are implemented. UWP offers TCP and UDP socket classes for client-server or peer-to-peer applications, whether connections are long-lived or an established connection is not required.
WebSockets	WebSockets provide a mechanism for fast, secure, two-way communication between a client and a server over the web using HTTP(S), and supporting both UTF-8 and binary messages.
HttpClient	Use Windows.Web.Http namespace API to send and receive information using the HTTP 2.0 and HTTP 1.1 protocols.
RSS/Atom feeds	Retrieve or create the most current and popular Web content using syndicated feeds generated according to the RSS and Atom standards using features in the Windows.Web.Syndication namespace.
Background transfers	Use the background transfer API to copy files reliably over the network.

Networking basics

Article • 10/20/2022

Things you must do for any network-enabled app.

Capabilities

In order to use networking, you must add appropriate capability elements to your app manifest. If no network capability is specified in your app's manifest, your app will have no networking capability, and any attempt to connect to the network will fail.

The following are the most-used networking capabilities.

Capability	Description
internetClient	Provides outbound access to the Internet and networks in public places, like airports and coffee shop. Most apps that require Internet access should use this capability.
internetClientServer	Gives the app inbound and outbound network access from the Internet and networks in public places like airports and coffee shops.
privateNetworkClientServer	Gives the app inbound and outbound network access at the user's trusted places, like home and work.

There are other capabilities that might be necessary for your app, in certain circumstances.

Capability	Description
enterpriseAuthentication	Allows an app to connect to network resources that require domain credentials. For example, an app that retrieves data from SharePoint servers on a private Intranet. With this capability your credentials can be used to access network resources on a network that requires credentials. An app with this capability can impersonate you on the network. You don't need this capability in order to allow your app to access the Internet via an authenticating proxy. For more details, see the documentation for the <i>Enterprise</i> capability scenario in Restricted capabilities.

Capability	Description
proximity	Required for near-field proximity communication with devices in close proximity to the computer. Near-field proximity may be used to send or connect with an application on a nearby device. This capability allows an app to access the network to connect to a device in close proximity, with user consent to send an invite or accept an invite.
sharedUserCertificates	This capability allows an app to access software and hardware certificates, such as smart card certificates. When this capability is invoked at runtime, the user must take action, such as inserting a card or selecting a certificate. With this capability, your software and hardware certificates or a smart card are used for identification in the app. This capability may be used by your employer, bank, or government services for identification.

Communicating when your app is not in the foreground

[Support your app with background tasks](#) contains general information about using background tasks to do work when your app is not in the foreground. More specifically, your code must take special steps to be notified when it is not the current foreground app and data arrives over the network for it. You used Control Channel Triggers for this purpose in Windows 8, and they are still supported in Windows 10. Full information about using Control Channel Triggers is available [here](#). A new technology in Windows 10 provides better functionality with lower overhead for some scenarios, such as push-enabled stream sockets: the socket broker and socket activity triggers.

If your app uses [DatagramSocket](#), [StreamSocket](#), or [StreamSocketListener](#), then your app can transfer ownership of an open socket to a socket broker provided by the system, and then leave the foreground, or even terminate. When a connection is made on the transferred socket, or traffic arrives on that socket, then your app or its designated background task are activated. If your app is not running, it is started. The socket broker then notifies your app using a [SocketActivityTrigger](#) that new traffic has arrived. Your app reclaims the socket from the socket broker and process the traffic on the socket. This means that your app consumes far less system resources when it is not actively processing network traffic.

The socket broker is intended to replace Control Channel Triggers where it is applicable, because it provides the same functionality, but with fewer restrictions and a smaller memory footprint. Socket broker can be used by apps that are not lock screen apps, and it is used the same way on phones as on other devices. Apps need not be running when traffic arrives in order to be activated by the socket broker. And the socket broker supports listening on TCP sockets, which Control Channel Triggers do not support.

Choosing a network trigger

There are some scenarios where either kind of trigger would be suitable. When you are choosing which kind of trigger to use in your app, consider the following advice.

- If you are using [IXMLHttpRequest2](#), [System.Net.Http.HttpClient](#) or [System.Net.Http.HttpClientHandler](#), you must use [ControlChannelTrigger](#).
- If you are using push-enabled **StreamSockets**, you can use control channel triggers, but should prefer [SocketActivityTrigger](#). The latter choice allows the system to free up memory and reduce power requirements when the connection is not being actively used.
- If you want to minimize the memory footprint of your app when it is not actively servicing network requests, prefer [SocketActivityTrigger](#) when possible.
- If you want your app to be able to receive data while the system is in Connected Standby mode, use [SocketActivityTrigger](#).

For details and examples of how to use the socket broker, see [Network communications in the background](#).

Secured connections

Secure Sockets Layer (SSL) and the more recent Transport Layer Security (TLS) are cryptographic protocols designed to provide authentication and encryption for network communication. These protocols are designed to prevent eavesdropping and tampering when sending and receiving network data. These protocols use a client-server model for the protocol exchanges. These protocols also use digital certificates and certificate authorities to verify that the server is who it claims to be.

Creating secure socket connections

A [StreamSocket](#) object can be configured to use SSL/TLS for communications between the client and the server. This support for SSL/TLS is limited to using the **StreamSocket** object as the client in the SSL/TLS negotiation. You cannot use SSL/TLS with the **StreamSocket** created by a [StreamSocketListener](#) when incoming communications are

received, because SSL/TLS negotiation as a server is not implemented by the **StreamSocket** class.

There are two ways to secure a **StreamSocket** connection with SSL/TLS:

- **ConnectAsync** - Make the initial connection to a network service and negotiate immediately to use SSL/TLS for all communications.
- **UpgradeToSslAsync** - Connect initially to a network service without encryption. The app may send or receive data. Then, upgrade the connection to use SSL/TLS for all further communications.

The **SocketProtectionLevel** specifies the desired socket protection level the app wants to establish or upgrade the connection with. However, the eventual protection level of the established connection is determined in a negotiation process between both endpoints of the connection. The result can be a lower protection level than the one you specified, if the other endpoint requests a lower level.

After the async operation has completed successfully, you can retrieve the requested protection level used in the **ConnectAsync** or **UpgradeToSslAsync** call through the **StreamSocketInformation.ProtectionLevel** property. However, this does not reflect the actual protection level the connection is using.

ⓘ Note

Your code should not implicitly depend on using a particular protection level, or on the assumption that a given security level is used by default. The security landscape changes constantly, and protocols and default protection levels change over time in order to avoid the use of protocols with known weaknesses. Defaults can vary depending on individual machine configuration, or on which software is installed and which patches have been applied. If your app depends on the use of a particular security level, then you must explicitly specify that level and then check to be sure that it is actually in use on the established connection.

Use ConnectAsync

ConnectAsync can be used to establish the initial connection with a network service and then negotiate immediately to use SSL/TLS for all communications. There are two **ConnectAsync** methods that support passing a *protectionLevel* parameter:

- **ConnectAsync(EndpointPair, SocketProtectionLevel)** - Starts an asynchronous operation on a **StreamSocket** object to connect to a remote network destination specified as an **EndpointPair** object and a **SocketProtectionLevel**.

- **ConnectAsync(HostName, String, SocketProtectionLevel)** - Starts an asynchronous operation on a **StreamSocket** object to connect to a remote destination specified by a remote hostname, a remote service name, and a **SocketProtectionLevel**.

If the *protectionLevel* parameter is set to

Windows.Networking.Sockets.SocketProtectionLevel.Ssl when calling either of the above **ConnectAsync** methods, the **StreamSocket** must will be established to use SSL/TLS for encryption. This value requires encryption and never allows a NULL cipher to be used.

The normal sequence to use with one of these **ConnectAsync** methods is the same.

- Create a **StreamSocket**.
- If an advanced option on the socket is needed, use the **StreamSocket.Control** property to get the **StreamSocketControl** instance associated with a **StreamSocket** object. Set a property on the **StreamSocketControl**.
- Call one of the above **ConnectAsync** methods to start an operation to connect to a remote destination and immediately negotiate the use of SSL/TLS.
- The SSL strength actually negotiated using **ConnectAsync** can be determined by getting the **StreamSocket.Information.ProtectionLevel** property after the async operation has completed successfully.

The following example creates a **StreamSocket** and tries to establish a connection to the network service and negotiate immediately to use SSL/TLS. If the negotiation is successful, all network communication using the **StreamSocket** between the client the network server will be encrypted.

C#

```
using Windows.Networking;
using Windows.Networking.Sockets;

// Define some variables and set values
StreamSocket clientSocket = new StreamSocket();

HostName serverHost = new HostName("www.contoso.com");
string serverServiceName = "https";

// For simplicity, the sample omits implementation of the
// NotifyUser method used to display status and error messages

// Try to connect to contoso using HTTPS (port 443)
try {

    // Call ConnectAsync method with SSL
    await clientSocket.ConnectAsync(serverHost, serverServiceName,
```



```

SocketProtectionLevel.Ssl);

    NotifyUser("Connected");
}
catch (Exception exception) {
    // If this is an unknown status it means that the error is fatal and
    // retry will likely fail.
    if (SocketError.GetStatus(exception.HResult) ==
SocketErrorStatus.Unknown) {
        throw;
    }

    NotifyUser("Connect failed with error: " + exception.Message);
    // Could retry the connection, but for this simple example
    // just close the socket.

    clientSocket.Dispose();
    clientSocket = null;
}

// Add code to send and receive data using the clientSocket
// and then close the clientSocket

```

C++/WinRT

```

#include <winrt/Windows.Networking.Sockets.h>

using namespace winrt;
...
// Define some variables, and set values.
Windows::Networking::Sockets::StreamSocket clientSocket;

Windows::Networking::HostName serverHost{ L"www.contoso.com" };
winrt::hstring serverServiceName{ L"https" };

// For simplicity, the sample omits implementation of the
// NotifyUser method used to display status and error messages.

// Try to connect to the server using HTTPS and SSL (port 443).
try
{
    co_await clientSocket.ConnectAsync(serverHost, serverServiceName,
Windows::Networking::Sockets::SocketProtectionLevel::Tls12);
    NotifyUser(L"Connected");
}
catch (winrt::hresult_error const& exception)
{
    NotifyUser(L"Connect failed with error: " + exception.message());
    clientSocket = nullptr;
}
// Add code to send and receive data using the clientSocket,
// then set the clientSocket to nullptr when done to close it.

```

C++

```
using Windows::Networking;
using Windows::Networking::Sockets;

// Define some variables and set values
StreamSocket^ clientSocket = new ref StreamSocket();

HostName^ serverHost = new ref HostName("www.contoso.com");
String serverServiceName = "https";

// For simplicity, the sample omits implementation of the
// NotifyUser method used to display status and error messages

// Try to connect to the server using HTTPS and SSL (port 443)
task<void>(clientSocket->ConnectAsync(serverHost, serverServiceName,
SocketProtectionLevel::SSL)).then([this] (task<void> previousTask) {
    try
    {
        // Try getting all exceptions from the continuation chain above
        this point.
        previousTask.Get();
        NotifyUser("Connected");
    }
    catch (Exception^ exception)
    {
        NotifyUser("Connect failed with error: " + exception->Message);

        clientSocket.Close();
        clientSocket = null;
    }
});
// Add code to send and receive data using the clientSocket
// Then close the clientSocket when done
```

Use UpgradeToSslAsync

When your code uses [UpgradeToSslAsync](#), it first establishes a connection to a network service without encryption. The app may send or receive some data, then upgrade the connection to use SSL/TLS for all further communications.

The [UpgradeToSslAsync](#) method takes two parameters. The *protectionLevel* parameter indicates the protection level desired. The *validationHostName* parameter is the hostname of the remote network destination that is used for validation when upgrading to SSL. Normally the *validationHostName* would be the same hostname that the app used to initially establish the connection. If the *protectionLevel* parameter is set to `Windows.System.Socket.SocketProtectionLevel.Ssl` when calling [UpgradeToSslAsync](#),

the [StreamSocket](#) must use the SSL/TLS for encryption on further communications over the socket. This value requires encryption and never allows a NULL cipher to be used.

The normal sequence to use with the [UpgradeToSslAsync](#) method is as follows:

- Create a [StreamSocket](#).
- If an advanced option on the socket is needed, use the [StreamSocket.Control](#) property to get the [StreamSocketControl](#) instance associated with a [StreamSocket](#) object. Set a property on the [StreamSocketControl](#).
- If any data needs to be sent and received unencrypted, send it now.
- Call the [UpgradeToSslAsync](#) method to start an operation to upgrade the connection to use SSL/TLS.
- The SSL strength actually negotiated using [UpgradeToSslAsync](#) can be determined by getting the [StreamSocketInformation.ProtectionLevel](#) property after the async operation completes successfully.

The following example creates a [StreamSocket](#), tries to establish a connection to the network service, sends some initial data, and then negotiates to use SSL/TLS. If the negotiation is successful, all network communication using the [StreamSocket](#) between the client and the network server will be encrypted.

C#

```
using Windows.Networking;
using Windows.Networking.Sockets;
using Windows.Storage.Streams;

// Define some variables and set values
StreamSocket clientSocket = new StreamSocket();

HostName serverHost = new HostName("www.contoso.com");
string serverServiceName = "http";

// For simplicity, the sample omits implementation of the
// NotifyUser method used to display status and error messages

// Try to connect to contoso using HTTP (port 80)
try {
    // Call ConnectAsync method with a plain socket
    await clientSocket.ConnectAsync(serverHost, serverServiceName,
SocketProtectionLevel.PlainSocket);

    NotifyUser("Connected");
}
catch (Exception exception) {
    // If this is an unknown status it means that the error is fatal and
    // retry will likely fail.
    if (SocketError.GetStatus(exception.HResult) ==
```

```

SocketErrorStatus.Unknown) {
    throw;
}

NotifyUser("Connect failed with error: " + exception.Message,
NotifyType.ErrorMessage);
// Could retry the connection, but for this simple example
// just close the socket.

clientSocket.Dispose();
clientSocket = null;
return;
}

// Now try to send some data
DataWriter writer = new DataWriter(clientSocket.OutputStream);
string hello = "Hello, World! @ ";
Int32 len = (int) writer.MeasureString(hello); // Gets the UTF-8 string
length.
writer.WriteInt32(len);
writer.WriteString(hello);
NotifyUser("Client: sending hello");

try {
    // Call StoreAsync method to store the hello message
    await writer.StoreAsync();

    NotifyUser("Client: sent data");

    writer.DetachStream(); // Detach stream, if not, DataWriter
destructor will close it.
}
catch (Exception exception) {
    NotifyUser("Store failed with error: " + exception.Message);
    // Could retry the store, but for this simple example
    // just close the socket.

    clientSocket.Dispose();
    clientSocket = null;
    return;
}

// Now upgrade the client to use SSL
try {
    // Try to upgrade to SSL
    await clientSocket.UpgradeToSslAsync(SocketProtectionLevel.Ssl,
serverHost);

    NotifyUser("Client: upgrade to SSL completed");

    // Add code to send and receive data
    // The close clientSocket when done
}
catch (Exception exception) {
    // If this is an unknown status it means that the error is fatal and

```

```

retry will likely fail.
    if (SocketError.GetStatus(exception.HResult) ==
SocketErrorStatus.Unknown) {
        throw;
    }

    NotifyUser("Upgrade to SSL failed with error: " +
exception.Message);

    clientSocket.Dispose();
    clientSocket = null;
    return;
}

```

C++/WinRT

```

#include <winrt/Windows.Networking.Sockets.h>
#include <winrt/Windows.Storage.Streams.h>

using namespace winrt;
using namespace Windows::Storage::Streams;
...
    // Define some variables, and set values.
    Windows::Networking::Sockets::StreamSocket clientSocket;

    Windows::Networking::HostName serverHost{ L"www.contoso.com" };
    winrt::hstring serverServiceName{ L"https" };

    // For simplicity, the sample omits implementation of the
    // NotifyUser method used to display status and error messages.

    // Try to connect to the server using HTTP (port 80).
    try
    {
        co_await clientSocket.ConnectAsync(serverHost, serverServiceName,
Windows::Networking::Sockets::SocketProtectionLevel::PlainSocket);
        NotifyUser(L"Connected");
    }
    catch (winrt::hresult_error const& exception)
    {
        NotifyUser(L"Connect failed with error: " + exception.message());
        clientSocket = nullptr;
    }

    // Now, try to send some data.
    DataWriter writer{ clientSocket.OutputStream() };
    winrt::hstring hello{ L"Hello, World! @ " };
    uint32_t len{ writer.MeasureString(hello) }; // Gets the size of the
string, in bytes.
    writer.WriteInt32(len);
    writer.WriteString(hello);
    NotifyUser(L"Client: sending hello");

```

```

try
{
    co_await writer.StoreAsync();
    NotifyUser(L"Client: sent hello");

    writer.DetachStream(); // Detach the stream when you want to
continue using it; otherwise, the DataWriter destructor closes it.
}
catch (winrt::hresult_error const& exception)
{
    NotifyUser(L"Store failed with error: " + exception.message());
    // We could retry the store operation. But, for this simple example,
just close the socket by setting it to nullptr.
    clientSocket = nullptr;
    co_return;
}

// Now, upgrade the client to use SSL.
try
{
    co_await
clientSocket.UpgradeToSslAsync(Windows::Networking::Sockets::SocketProtectio
nLevel::Tls12, serverHost);
    NotifyUser(L"Client: upgrade to SSL completed");

    // Add code to send and receive data using the clientSocket,
    // then set the clientSocket to nullptr when done to close it.
}
catch (winrt::hresult_error const& exception)
{
    // If this is an unknown status, then the error is fatal and retry
will likely fail.
    Windows::Networking::Sockets::SocketErrorStatus socketErrorStatus{
Windows::Networking::Sockets::SocketError::GetStatus(exception.to_abi()) };
    if (socketErrorStatus ==
Windows::Networking::Sockets::SocketErrorStatus::Unknown)
    {
        throw;
    }

    NotifyUser(L"Upgrade to SSL failed with error: " +
exception.message());
    // We could retry the store operation. But for this simple example,
just close the socket by setting it to nullptr.
    clientSocket = nullptr;
    co_return;
}
}

```

C++

```

using Windows::Networking;
using Windows::Networking::Sockets;
using Windows::Storage::Streams;

```

```

// Define some variables and set values
StreamSocket^ clientSocket = new ref StreamSocket();

Hostname^ serverHost = new ref HostName("www.contoso.com");
String serverServiceName = "http";

// For simplicity, the sample omits implementation of the
// NotifyUser method used to display status and error messages

// Try to connect to contoso using HTTP (port 80)
task<void>(clientSocket->ConnectAsync(serverHost, serverServiceName,
SocketProtectionLevel::PlainSocket)).then([this] (task<void> previousTask) {
    try
    {
        // Try getting all exceptions from the continuation chain above
        this point.
        previousTask.Get();
        NotifyUser("Connected");
    }
    catch (Exception^ exception)
    {
        NotifyUser("Connect failed with error: " + exception->Message);

        clientSocket->Close();
        clientSocket = null;
    }
});

// Now try to send some data
DataWriter^ writer = new ref DataWriter(clientSocket.OutputStream);
String hello = "Hello, World! @ ";
Int32 len = (int) writer->MeasureString(hello); // Gets the UTF-8 string
length.
writer->writeInt32(len);
writer->writeString(hello);
NotifyUser("Client: sending hello");

task<void>(writer->StoreAsync()).then([this] (task<void> previousTask) {
    try {
        // Try getting all exceptions from the continuation chain above
        this point.
        previousTask.Get();

        NotifyUser("Client: sent hello");

        writer->DetachStream(); // Detach stream, if not, DataWriter
        destructor will close it.
    }
    catch (Exception^ exception) {
        NotifyUser("Store failed with error: " + exception->Message);
        // Could retry the store, but for this simple example
        // just close the socket.

        clientSocket->Close();
    }
});

```

```

        clientSocket = null;
        return
    }
});

// Now upgrade the client to use SSL
task<void>(clientSocket->UpgradeToSslAsync(clientSocket.SocketProtectionLevel.Ssl,
serverHost)).then([this] (task<void> previousTask) {
    try {
        // Try getting all exceptions from the continuation chain above
        this point.
        previousTask.Get();

        NotifyUser("Client: upgrade to SSL completed");

        // Add code to send and receive data
        // Then close clientSocket when done
    }
    catch (Exception^ exception) {
        // If this is an unknown status it means that the error is fatal
        and retry will likely fail.
        if (SocketError.GetStatus(exception.HResult) ==
        SocketErrorStatus.Unknown) {
            throw;
        }

        NotifyUser("Upgrade to SSL failed with error: " +
        exception.Message);

        clientSocket->Close();
        clientSocket = null;
        return;
    }
});

```

Creating secure WebSocket connections

Like traditional socket connections, WebSocket connections can also be encrypted with Transport Layer Security (TLS)/Secure Sockets Layer (SSL) when using the [StreamWebSocket](#) and [MessageWebSocket](#) features for a UWP app. In most cases you'll want to use a secure WebSocket connection. This will increase the chances that your connection will succeed, as many proxies will reject unencrypted WebSocket connections.

For examples of how to create, or upgrade to, a secure socket connection to a network service, see [How to secure WebSocket connections with TLS/SSL](#).

In addition to TLS/SSL encryption, a server may require a **Sec-WebSocket-Protocol** header value to complete the initial handshake. This value, represented by the [StreamWebSocketInformation.Protocol](#) and [MessageWebSocketInformation.Protocol](#) properties, indicate the protocol version of the connection and enables the server to correctly interpret the opening handshake and the data being exchanged afterwards. Using this protocol information, if at any point if the server cannot interpret the incoming data in a safe manner the connection can be closed.

If the initial request from the client either does not contain this value, or provides a value that doesn't match what the server expects, the expected value is sent from the server to the client on WebSocket handshake error.

Authentication

How to provide authentication credentials when connecting over the network.

Providing a client certificate with the StreamSocket class

The [Windows.Networking.Sockets.StreamSocket](#) class supports using SSL/TLS to authenticate the server the app is talking to. In certain cases, the app also needs to authenticate itself to the server using a TLS client certificate. In Windows 10, you can provide a client certificate on the [StreamSocket.Control](#) object (this must be set before the TLS handshake is started). If the server requests the client certificate, Windows will respond with the certificate provided.

Here is a code snippet showing how to implement this:

C#

```
var socket = new StreamSocket();
Windows.Security.Cryptography.Certificates.Certificate certificate = await
GetClientCert();
socket.Control.ClientCertificate = certificate;
await socket.ConnectAsync(destination, SocketProtectionLevel.Tls12);
```

Providing authentication credentials to a web service

The networking APIs that enable apps to interact with secure web services each provide their own methods to either initialize a client or set a request header with server and proxy authentication credentials. Each method is set with a [PasswordCredential](#) object that indicates a user name, password, and the resource for which these credentials are used. The following table provides a mapping of these APIs:

WebSockets	MessageWebSocketControl.ServerCredential
	MessageWebSocketControl.ProxyCredential
	StreamWebSocketControl.ServerCredential
	StreamWebSocketControl.ProxyCredential
Background Transfer	BackgroundDownloader.ServerCredential
	BackgroundDownloader.ProxyCredential
	BackgroundUploader.ServerCredential
	BackgroundUploader.ProxyCredential
Syndication	SyndicationClient(PasswordCredential)
	SyndicationClient.ServerCredential
	SyndicationClient.ProxyCredential
AtomPub	AtomPubClient(PasswordCredential)
	AtomPubClient.ServerCredential
	AtomPubClient.ProxyCredential

Handling network exceptions

In most areas of programming, an exception indicates a significant problem or failure, caused by some flaw in the program. In network programming, there is an additional source for exceptions: the network itself, and the nature of network communications. Network communications are inherently unreliable and prone to unexpected failure. For each of the ways your app uses networking, you must maintain some state information; and your app code must handle network exceptions by updating that state information and initiating appropriate logic for your app to re-establish or retry communication failures.

When Universal Windows apps throw an exception, your exception handler can retrieve more detailed information on the cause of the exception to better understand the failure and make appropriate decisions.

Each language projection supports a method to access this more detailed information. An exception projects as an **HRESULT** value in Universal Windows apps. The *Winerror.h* include file contains a very large list of possible **HRESULT** values that includes network errors.

The networking APIs support different methods for retrieving this detailed information on the cause of an exception.

- Some APIs provide a helper method that converts the **HRESULT** value from the exception to an enumeration value.
- Other APIs provide a method to retrieve the actual **HRESULT** value.

Related topics

- [Networking API Improvements in Windows 10](#) 

Which networking technology?

Article • 10/20/2022

An overview of the networking technologies available for a UWP developer, with suggestions on how to choose the technologies that are right for your app.

Sockets

Use [Sockets](#) when you are communicating with another device and want to use your own protocol.

Two implementations of sockets are available for Universal Windows Platform (UWP) developers: [Windows.Networking.Sockets](#), and [Winsock](#). If you are writing new code, then Windows.Networking.Sockets has the advantage of being a modern API, designed for use by UWP developers. If you are using cross-platform networking libraries or other existing Winsock code, or prefer the Winsock API, then use that.

When to use sockets

- Both sockets implementations enable you to communicate with other devices using protocols of your own choice, using TCP or UDP.
- Choose the sockets API that best meets your needs based on experience and any existing code you might be using.

When not to use sockets

- Don't implement your own HTTP(S) stack using sockets. Use [HttpClient](#) instead.
- If WebSockets (the [StreamWebSocket](#) and [MessageWebSocket](#) classes) meet your communications needs (TCP to/from a web server), consider using them rather than spend your own time and development resources implementing similar functionality with sockets.

Websockets

The [WebSockets](#) protocol defines a mechanism for fast, secure two-way communication between a client and a server over the web. Data is transferred immediately over a full-duplex single socket connection, allowing messages to be sent and received from both endpoints in real time. WebSockets are ideal for use in real-time gaming where instant

social network notifications and up-to-date displays of information (like game statistics) need to be secure and use fast data transfer. UWP developers can use the [StreamWebSocket](#) and [MessageWebSocket](#) classes to connect with servers that support the WebSocket protocol.

When to use Websockets

- When you want to send and receive data on an ongoing basis between a device and a server.

When not to use Websockets

- If you are sending or receiving data infrequently, you might find it simpler to make individual HTTP requests from the device to the server, rather than establish and maintain a WebSocket connection.
- WebSockets may not be suitable for very high-volume situations. Consider modeling your data flows and simulating your traffic through WebSockets before committing to using them in your design.

HttpClient

Use [HttpClient](#) (and the rest of the [Windows.Web.Http](#) namespace API) when you are using HTTP(S) to communicate with a web service or a web server.

When to use HttpClient

- When using HTTP(S) to communicate with web services.
- When uploading or downloading a small number of smaller files.
- If WebSockets (the [StreamWebSocket](#) and [MessageWebSocket](#) classes) meet your communications needs (TCP to/from a web server), and the web server in question supports WebSockets, consider using them rather than spend your own time and development resources implementing similar functionality with HttpClient.
- When you are streaming content over the network.

When not to use HttpClient

- If you are transferring large files, or large numbers of files, consider using background transfers instead.
- If you want to be able to restrict upload/download limits based on connection type, or if you want to save progress and resume upload/download after an

interruption, you must use background transfers.

- If you are communicating between two devices and neither one is designed to act as an HTTP(S) server, you should use sockets. Do not attempt to implement your own HTTP server and use [HttpClient](#) to communicate with it.

Background transfers

Use the [background transfer API](#) when you want to reliably transfer files over the network. The background transfer API provides advanced upload and download features that run in the background during app suspension and persist beyond app termination. The API monitors network status and automatically suspends and resumes transfers when connectivity is lost, and transfers are also Data Sense-aware and Battery Sense-aware, meaning that download activity adjusts based on your current connectivity and device battery status. These capabilities are essential when your app is running on mobile or battery-powered devices. The API is ideal for uploading and downloading large files using HTTP(S). FTP is also supported, but only for downloads.

A new background transfer feature in Windows 10 is the ability to trigger post-processing when a file transfer has completed, so that you can update local catalogs, activate other apps, or notify the user when a download is complete.

When to use background transfers

- Use background transfers to reliably transfer large files, or large numbers of files.
- Use background transfers with background transfer completion groups when you want to post-process file transfers with a background task.
- Use background transfers if you want to be able to resume a transfer in progress after a network interruption.
- Use background transfers if you want to be able to change transfer behavior based on network conditions like being on a metered data plan.

When not to use background transfers

- If you are transferring a small number of small files, and you don't need to do any post-processing when the transfer is complete, consider using [HttpClient](#) PUT or POST methods.
- If you want to stream data and use it locally as it arrives, use [HttpClient](#).

Additional network-related technologies

Connection quality

The APIs in the [Windows.Networking.Connectivity](#) namespace enable you to access network connectivity, cost, and usage information. For more information about using this API, see [Accessing network connection state and managing network costs](#).

DNS Service Discovery

The [Windows.Networking.ServiceDiscovery.Dnssd](#) API enables you to advertise a network service to other devices on the network using the DNS-SD protocol described in IETF [RFC 2782](#).

Communicating over Bluetooth

Among other things, the [Windows.Devices.Bluetooth](#) API enables you to use Bluetooth to connect to other devices and transfer data. For more information, see [Send or receive files with RFCOMM](#).

Push notifications (WNS)

The [Windows.Networking.PushNotifications](#) API enables you to use the Windows Notification Service (WNS) to receive push notifications over the network. For more information about using this API, see [Windows Push Notification Services \(WNS\) overview](#)

Near field communications

The [Windows.Networking.Proximity](#) API enables you to use near-field communications for apps that use proximity or tap with devices to enable easy data transfer. For more information about using this API, see [Proximity and tapping](#).

RSS/Atom feeds

The [Windows.Web.Syndication](#) API enables you to manage syndication feeds using RSS and Atom formats. For more information about using this API, see [RSS/Atom feeds](#).

Wi-Fi enumeration and connection control

The [Windows.Devices.WiFi](#) API enables you to enumerate Wi-Fi adapters, scan for available Wi-Fi networks, and connect an adapter to a network.

Radio control

The [Windows.Devices.Radios](#) API allows you to find and control radios on the local device, including Wi-Fi and Bluetooth.

Wi-Fi Direct

The [Windows.Devices.WiFiDirect](#) API allows you to connect and communicate with other local devices using Wi-Fi Direct to create ad-hoc local wireless networks.

Wi-Fi Direct services

The [Windows.Devices.WiFiDirect.Services](#) API enables you to provide Wi-Fi Direct services and connect to them. Wi-Fi Direct Services are the way that one device on a Wi-Fi direct ad-hoc network (a Service Advertiser) offers capabilities to another device (a Service Seeker) over a Wi-Fi Direct connection.

Mobile operators

Windows 10 exposes to a wide developer audience some APIs that have previously only been exposed to device manufacturers and mobile operators. Note that while these APIs are exposed now, they are also gated by specific app capabilities that must be approved by Microsoft before an app can be published. Actual use of these APIs will still be limited primarily to device manufacturers and mobile operators.

Network operations

The [Windows.Networking.NetworkOperators](#) API deals primarily with the configuration and provisioning of phones. As such, permission to use the capabilities that control it are limited to device manufacturers and telecom providers.

SMS

The [Windows.Devices.Sms](#) namespace deals with SMS and related messages as low-level entities. It is provided for use by mobile operators for app-directed SMS use, and is controlled by a capability that will not be approved for use by most app developers. If you are writing an app to deal with messages, you should use the [Windows.ApplicationModel.Chat](#) API instead, as it is designed to handle not just SMS messages, but also messages from other sources such as realtime chat apps, enabling a much richer chat/messaging experience.

Network communications in the background

Article • 10/20/2022

To continue network communication while it's not in the foreground, your app can use background tasks and one of these two options.

- **Socket broker.** If your app uses sockets for long-term connections then, when it leaves the foreground, it can delegate ownership of a socket to a system socket broker. The broker then: activates your app when traffic arrives on the socket; transfers ownership back to your app; and your app then processes the arriving traffic.
- **Control channel triggers.**

Performing network operations in background tasks

- Use a [SocketActivityTrigger](#) to activate the background task when a packet is received and you need to perform a short-lived task. After performing the task, the background task should terminate in order to save power.
- Use a [ControlChannelTrigger](#) to activate the background task when a packet is received and you need to perform a long-lived task.

Network-related conditions and flags

- Add the **InternetAvailable** condition to your background task [BackgroundTaskBuilder.AddCondition](#) to delay triggering the background task until the network stack is running. This condition saves power because the background task won't execute until the network is up. This condition does not provide real-time activation.

Regardless of the trigger you use, set [IsNetworkRequested](#) on your background task to ensure that the network stays up while the background task runs. This tells the background task infrastructure to keep the network up while the task is executing, even if the device has entered Connected Standby mode. If your background task does not use **IsNetworkRequested**, then your background task will not be able to access the network when in Connected Standby mode (for example, when a phone's screen is turned off).

Socket broker and the SocketActivityTrigger

If your app uses [DatagramSocket](#), [StreamSocket](#), or [StreamSocketListener](#) connections, then you should use [SocketActivityTrigger](#) and the socket broker to be notified when traffic arrives for your app while it's not in the foreground.

In order for your app to receive and process data received on a socket when your app is not active, your app must perform some one-time setup at startup, and then transfer socket ownership to the socket broker when it is transitioning to a state where it is not active.

The one-time setup steps are to create a trigger, to register a background task for the trigger, and to enable the socket for the socket broker:

- Create a **SocketActivityTrigger** and register a background task for the trigger with the `TaskEntryPoint` parameter set to your code for processing a received packet.

C#

```
var socketTaskBuilder = new BackgroundTaskBuilder();
socketTaskBuilder.Name = _backgroundTaskName;
socketTaskBuilder.TaskEntryPoint = _backgroundTaskEntryPoint;
var trigger = new SocketActivityTrigger();
socketTaskBuilder.SetTrigger(trigger);
_task = socketTaskBuilder.Register();
```

- Call **EnableTransferOwnership** on the socket, before you bind the socket.

C#

```
_tcpListener = new StreamSocketListener();

// Note that EnableTransferOwnership() should be called before
bind,
// so that tcpip keeps required state for the socket to enable
connected
// standby action. Background task Id is taken as a parameter to
tie wake pattern
// to a specific background task.
_tcpListener.
EnableTransferOwnership(_task.TaskId, SocketActivityConnectedStandbyAction.Wake);

_tcpListener.ConnectionReceived += OnConnectionReceived;
await _tcpListener.BindServiceNameAsync("my-service-name");
```

Once your socket is properly set up, when your app is about to suspend, call **TransferOwnership** on the socket to transfer it to a socket broker. The broker monitors

the socket and activates your background task when data is received. The following example includes a utility **TransferOwnership** function to perform the transfer for **StreamSocketListener** sockets. (Note that the different types of sockets each have their own **TransferOwnership** method, so you must call the method appropriate for the socket whose ownership you are transferring. Your code would probably contain an overloaded **TransferOwnership** helper with one implementation for each socket type you use, so that the **OnSuspending** code remains easy to read.)

An app transfers ownership of a socket to a socket broker and passes the ID for the background task using the appropriate one of the following methods:

- One of the **TransferOwnership** methods on a **DatagramSocket**.
- One of the **TransferOwnership** methods on a **StreamSocket**.
- One of the **TransferOwnership** methods on a **StreamSocketListener**.

C#

```
// declare int _transferOwnershipCount as a field.

private void TransferOwnership(StreamSocketListener tcpListener)
{
    await tcpListener.CancelIOAsync();

    var dataWriter = new DataWriter();
    ++_transferOwnershipCount;
    dataWriter.WriteInt32(transferOwnershipCount);
    var context = new SocketActivityContext(dataWriter.DetachBuffer());
    tcpListener.TransferOwnership(_socketId, context);
}

private void OnSuspending(object sender, SuspendingEventArgs e)
{
    var deferral = e.SuspendingOperation.GetDeferral();

    TransferOwnership(_tcpListener);
    deferral.Complete();
}
```

In your background task's event handler:

- First, get a background task deferral so that you can handle the event using asynchronous methods.

C#

```
var deferral = taskInstance.GetDeferral();
```

- Next, extract the `SocketActivityTriggerDetails` from the event arguments, and find the reason that the event was raised:

C#

```
var details = taskInstance.TriggerDetails as SocketActivityTriggerDetails;
var socketInformation = details.SocketInformation;
switch (details.Reason)
```

- If the event was raised because of socket activity, create a `DataReader` on the socket, load the reader asynchronously, and then use the data according to your app's design. Note that you must return ownership of the socket back to the socket broker, in order to be notified of further socket activity again.

In the following example, the text received on the socket is displayed in a toast.

C#

```
case SocketActivityTriggerReason.SocketActivity:
    var socket = socketInformation.StreamSocket;
    DataReader reader = new DataReader(socket.InputStream);
    reader.InputStreamOptions = InputStreamOptions.Partial;
    await reader.LoadAsync(250);
    var dataString =
reader.ReadString(reader.UnconsumedBufferLength);
    ShowToast(dataString);
    socket.TransferOwnership(socketInformation.Id); /* Important! */
    break;
```

- If the event was raised because a keep alive timer expired, then your code should send some data over the socket in order to keep the socket alive and restart the keep alive timer. Again, it is important to return ownership of the socket back to the socket broker in order to receive further event notifications:

C#

```
case SocketActivityTriggerReason.KeepAliveTimerExpired:
    socket = socketInformation.StreamSocket;
    DataWriter writer = new DataWriter(socket.OutputStream);
    writer.WriteBytes(Encoding.UTF8.GetBytes("Keep alive"));
    await writer.StoreAsync();
    writer.DetachStream();
    writer.Dispose();
    socket.TransferOwnership(socketInformation.Id); /* Important! */
    break;
```

- If the event was raised because the socket was closed, re-establish the socket, making sure that after you create the new socket, you transfer ownership of it to the socket broker. In this sample, the hostname and port are stored in local settings so that they can be used to establish a new socket connection:

C#

```
case SocketActivityTriggerReason.SocketClosed:
    socket = new StreamSocket();
    socket.EnableTransferOwnership(taskInstance.Task.TaskId,
    SocketActivityConnectedStandbyAction.Wake);
    if (ApplicationData.Current.LocalSettings.Values["hostname"] ==
    null)
    {
        break;
    }
    var hostname =
    (String)ApplicationData.Current.LocalSettings.Values["hostname"];
    var port =
    (String)ApplicationData.Current.LocalSettings.Values["port"];
    await socket.ConnectAsync(new HostName(hostname), port);
    socket.TransferOwnership(socketId);
    break;
```

- Don't forget to Complete your deferral, once you have finished processing the event notification:

C#

```
deferral.Complete();
```

For a complete sample demonstrating the use of the [SocketActivityTrigger](#) and socket broker, see the [SocketActivityStreamSocket sample](#) [↗](#). The initialization of the socket is performed in Scenario1_Connect.xaml.cs, and the background task implementation is in SocketActivityTask.cs.

You will probably notice that the sample calls **TransferOwnership** as soon as it creates a new socket or acquires an existing socket, rather than using the **OnSuspending** even handler to do so as described in this topic. This is because the sample focuses on demonstrating the [SocketActivityTrigger](#), and doesn't use the socket for any other activity while it is running. Your app will probably be more complex, and should use **OnSuspending** to determine when to call **TransferOwnership**.

Control channel triggers

First, ensure that you're using control channel triggers (CCTs) appropriately. If you're using [DatagramSocket](#), [StreamSocket](#), or [StreamSocketListener](#) connections, then we recommend that you use [SocketActivityTrigger](#). You can use CCTs for [StreamSocket](#), but they use more resources and might not work in Connected Standby mode.

If you are using WebSockets, [IXMLHttpRequest2](#), [System.Net.Http.HttpClient](#), or [Windows.Web.Http.HttpClient](#), then you must use [ControlChannelTrigger](#).

ControlChannelTrigger with WebSockets

Important

The feature described in this section ([ControlChannelTrigger with WebSockets](#)) is supported in version 10.0.15063.0 of the SDK, and earlier. It is also supported in pre-release versions of the [Windows 10 Insider Preview](#) .

Some special considerations apply when using [MessageWebSocket](#) or [StreamWebSocket](#) with [ControlChannelTrigger](#). There are some transport-specific usage patterns and best practices that should be followed when using a [MessageWebSocket](#) or [StreamWebSocket](#) with [ControlChannelTrigger](#). In addition, these considerations affect the way that requests to receive packets on the [StreamWebSocket](#) are handled. Requests to receive packets on the [MessageWebSocket](#) are not affected.

The following usage patterns and best practices should be followed when using [MessageWebSocket](#) or [StreamWebSocket](#) with [ControlChannelTrigger](#):

- An outstanding socket receive must be kept posted at all times. This is required to allow the push notification tasks to occur.
- The WebSocket protocol defines a standard model for keep-alive messages. The [WebSocketKeepAlive](#) class can send client-initiated WebSocket protocol keep-alive messages to the server. The [WebSocketKeepAlive](#) class should be registered as the TaskEntryPoint for a KeepAliveTrigger by the app.

Some special considerations affect the way that requests to receive packets on the [StreamWebSocket](#) are handled. In particular, when using a [StreamWebSocket](#) with the [ControlChannelTrigger](#), your app must use a raw async pattern for handling reads instead of the `await` model in C# and VB.NET or Tasks in C++. The raw async pattern is illustrated in a code sample later in this section.

Using the raw async pattern allows Windows to synchronize the `IBackgroundTask.Run` method on the background task for the `ControlChannelTrigger` with the return of the receive completion callback. The `Run` method is invoked after the completion callback returns. This ensures that the app has received the data/errors before the `Run` method is invoked.

It is important to note that the app has to post another read before it returns control from the completion callback. It is also important to note that the `DataReader` cannot be directly used with the `MessageWebSocket` or `StreamWebSocket` transport since that breaks the synchronization described above. It is not supported to use the `DataReader.LoadAsync` method directly on top of the transport. Instead, the `IBuffer` returned by the `IInputStream.ReadAsync` method on the `StreamWebSocket.InputStream` property can be later passed to `DataReader.FromBuffer` method for further processing.

The following sample shows how to use a raw async pattern for handling reads on the [StreamWebSocket](#).

C#

```

void PostSocketRead(int length)
{
    try
    {
        var readBuf = new Windows.Storage.Streams.Buffer((uint)length);
        var readOp = socket.InputStream.ReadAsync(readBuf, (uint)length,
InputStreamOptions.Partial);
        readOp.Completed = (IAsyncOperationWithProgress<IBuffer, uint>
        asyncAction, AsyncStatus asyncStatus) =>
        {
            switch (asyncStatus)
            {
                case AsyncStatus.Completed:
                case AsyncStatus.Error:
                    try
                    {
                        // GetResults in AsyncStatus::Error is called as it
throws a user friendly error string.
                        IBuffer localBuf = asyncAction.GetResults();
                        uint bytesRead = localBuf.Length;
                        readPacket = DataReader.FromBuffer(localBuf);
                        OnDataReadCompletion(bytesRead, readPacket);
                    }
                    catch (Exception exp)
                    {
                        Diag.DebugPrint("Read operation failed: " +
exp.Message);
                    }
                    break;
            }
        }
    }
}

```

```

        case AsyncStatus.Canceled:

            // Read is not cancelled in this sample.
            break;

        }
    };
}
catch (Exception exp)
{
    Diag.DebugPrint("failed to post a read failed with error: " +
exp.Message);
}
}

```

The read completion handler is guaranteed to fire before the [IBackgroundTask.Run](#) method on the background task for the [ControlChannelTrigger](#) is invoked. Windows has internal synchronization to wait for an app to return from the read completion callback. The app typically quickly processes the data or the error from the [MessageWebSocket](#) or [StreamWebSocket](#) in the read completion callback. The message itself is processed within the context of the [IBackgroundTask.Run](#) method. In this sample below, this point is illustrated by using a message queue that the read completion handler inserts the message into and the background task later processes.

The following sample shows the read completion handler to use with a raw async pattern for handling reads on the [StreamWebSocket](#).

C#

```

public void OnDataReadCompletion(uint bytesRead, DataReader readPacket)
{
    if (readPacket == null)
    {
        Diag.DebugPrint("DataReader is null");

        // Ideally when read completion returns error,
        // apps should be resilient and try to
        // recover if there is an error by posting another recv
        // after creating a new transport, if required.
        return;
    }
    uint buffLen = readPacket.UnconsumedBufferLength;
    Diag.DebugPrint("bytesRead: " + bytesRead + ", unconsumedbufflength: " +
buffLen);

    // check if buffLen is 0 and treat that as fatal error.
    if (buffLen == 0)
    {
        Diag.DebugPrint("Received zero bytes from the socket. Server must
have closed the connection.");
        Diag.DebugPrint("Try disconnecting and reconnecting to the server");
    }
}

```



```

        return;
    }

    // Perform minimal processing in the completion
    string message = readPacket.ReadString(buffLen);
    Diag.DebugPrint("Received Buffer : " + message);

    // Enqueue the message received to a queue that the push notify
    // task will pick up.
    AppContext.messageQueue.Enqueue(message);

    // Post another receive to ensure future push notifications.
    PostSocketRead(MAX_BUFFER_LENGTH);
}

```

An additional detail for Websockets is the keep-alive handler. The WebSocket protocol defines a standard model for keep-alive messages.

When using [MessageWebSocket](#) or [StreamWebSocket](#), register a [WebSocketKeepAlive](#) class instance as the [TaskEntryPoint](#) for a [KeepAliveTrigger](#) to allow the app to be unsuspended and send keep-alive messages to the server (remote endpoint) periodically. This should be done as part of the background registration app code as well as in the package manifest.

This task entry point of [Windows.Sockets.WebSocketKeepAlive](#) needs to be specified in two places:

- When creating [KeepAliveTrigger](#) trigger in the source code (see example below).
- In the app package manifest for the keepalive background task declaration.

The following sample adds a network trigger notification and a keepalive trigger under the <Application> element in an app manifest.

XML

```

<Extensions>
  <Extension Category="windows.backgroundTasks"
    Executable="$targetnametoken$.exe"
    EntryPoint="Background.PushNotifyTask">
    <BackgroundTasks>
      <Task Type="controlChannel" />
    </BackgroundTasks>
  </Extension>
  <Extension Category="windows.backgroundTasks"
    Executable="$targetnametoken$.exe"
    EntryPoint="Windows.Networking.Sockets.WebSocketKeepAlive">
    <BackgroundTasks>
      <Task Type="controlChannel" />
    </BackgroundTasks>
  </Extension>
</Extensions>

```

```
</Extension>
</Extensions>
```

An app must be extremely careful when using an `await` statement in the context of a `ControlChannelTrigger` and an asynchronous operation on a `StreamWebSocket`, `MessageWebSocket`, or `StreamSocket`. A `Task<bool>` object can be used to register a `ControlChannelTrigger` for push notification and WebSocket keep-alives on the `StreamWebSocket` and connect the transport. As part of the registration, the `StreamWebSocket` transport is set as the transport for the `ControlChannelTrigger` and a read is posted. The `Task.Result` will block the current thread until all steps in the task execute and return statements in message body. The task is not resolved until the method returns either true or false. This guarantees that the whole method is executed. The `Task` can contain multiple `await` statements that are protected by the `Task`. This pattern should be used with the `ControlChannelTrigger` object when a `StreamWebSocket` or `MessageWebSocket` is used as the transport. For those operations that may take a long period of time to complete (a typical async read operation, for example), the app should use the raw async pattern discussed previously.

The following sample registers `ControlChannelTrigger` for push notification and WebSocket keep-alives on the `StreamWebSocket`.

C#

```
private bool RegisterWithControlChannelTrigger(string serverUri)
{
    // Make sure the objects are created in a system thread
    // Demonstrate the core registration path
    // Wait for the entire operation to complete before returning from this
    // method.
    // The transport setup routine can be triggered by user control, by
    // network state change
    // or by keepalive task
    Task<bool> registerTask = RegisterWithCCTHelper(serverUri);
    return registerTask.Result;
}

async Task<bool> RegisterWithCCTHelper(string serverUri)
{
    bool result = false;
    socket = new StreamWebSocket();

    // Specify the keepalive interval expected by the server for this app
    // in order of minutes.
    const int serverKeepAliveInterval = 30;

    // Specify the channelId string to differentiate this
    // channel instance from any other channel instance.
    // When background task fires, the channel object is provided
```

```

// as context and the channel id can be used to adapt the behavior
// of the app as required.
const string channelId = "channelOne";

// For websockets, the system does the keepalive on behalf of the app
// But the app still needs to specify this well known keepalive task.
// This should be done here in the background registration as well
// as in the package manifest.
const string WebSocketKeepAliveTask =
"Windows.Networking.Sockets.WebSocketKeepAlive";

// Try creating the controlchanneltrigger if this has not been already
// created and stored in the property bag.
ControlChannelTriggerStatus status;

// Create the ControlChannelTrigger object and request a hardware slot
for this app.
// If the app is not on LockScreen, then the ControlChannelTrigger
constructor will
// fail right away.
try
{
    channel = new ControlChannelTrigger(channelId,
serverKeepAliveInterval,
ControlChannelTriggerResourceType.RequestHardwareSlot);
}
catch (UnauthorizedAccessException exp)
{
    Diag.DebugPrint("Is the app on lockscreen? " + exp.Message);
    return result;
}

Uri serverUriInstance;
try
{
    serverUriInstance = new Uri(serverUri);
}
catch (Exception exp)
{
    Diag.DebugPrint("Error creating URI: " + exp.Message);
    return result;
}

// Register the apps background task with the trigger for keepalive.
var keepAliveBuilder = new BackgroundTaskBuilder();
keepAliveBuilder.Name = "KeepaliveTaskForChannelOne";
keepAliveBuilder.TaskEntryPoint = WebSocketKeepAliveTask;
keepAliveBuilder.SetTrigger(channel.KeepAliveTrigger);
keepAliveBuilder.Register();

// Register the apps background task with the trigger for push
notification task.
var pushNotifyBuilder = new BackgroundTaskBuilder();
pushNotifyBuilder.Name = "PushNotificationTaskForChannelOne";

```

```

pushNotifyBuilder.TaskEntryPoint = "Background.PushNotifyTask";
pushNotifyBuilder.SetTrigger(channel.PushNotificationTrigger);
pushNotifyBuilder.Register();

// Tie the transport method to the ControlChannelTrigger object to push
enable it.
// Note that if the transport's TCP connection is broken at a later
point of time,
// the ControlChannelTrigger object can be reused to plug in a new
transport by
// calling UsingTransport API again.
try
{
    channel.UsingTransport(socket);

    // Connect the socket
    //
    // If connect fails or times out it will throw exception.
    // ConnectAsync can also fail if hardware slot was requested
    // but none are available
    await socket.ConnectAsync(serverUriInstance);

    // Call WaitForPushEnabled API to make sure the TCP connection has
    // been established, which will mean that the OS will have allocated
    // any hardware slot for this TCP connection.
    //
    // In this sample, the ControlChannelTrigger object was created by
    // explicitly requesting a hardware slot.
    //
    // On systems that without connected standby, if app requests
hardware slot as above,
    // the system will fallback to a software slot automatically.
    //
    // On systems that support connected standby,, if no hardware slot
is available, then app
    // can request a software slot by re-creating the
ControlChannelTrigger object.
    status = channel.WaitForPushEnabled();
    if (status != ControlChannelTriggerStatus.HardwareSlotAllocated
        && status != ControlChannelTriggerStatus.SoftwareSlotAllocated)
    {
        throw new Exception(string.Format("Neither hardware nor software
slot could be allocated. ChannelStatus is {0}", status.ToString()));
    }

    // Store the objects created in the property bag for later use.
    CoreApplication.Properties.Remove(channel.ControlChannelTriggerId);

    var appContext = new AppContext(this, socket, channel,
channel.ControlChannelTriggerId);
    ((IDictionary<string,
object>)CoreApplication.Properties).Add(channel.ControlChannelTriggerId,
appContext);
    result = true;
}

```

```

        // Almost done. Post a read since we are using streamwebsocket
        // to allow push notifications to be received.
        PostSocketRead(MAX_BUFFER_LENGTH);
    }
    catch (Exception exp)
    {
        Diag.DebugPrint("RegisterWithCCTHelper Task failed with: " +
exp.Message);

        // Exceptions may be thrown for example if the application has not
        // registered the background task class id for using real time
        communications
        // broker in the package manifest.
    }
    return result
}

```

For more information on using [MessageWebSocket](#) or [StreamWebSocket](#) with [ControlChannelTrigger](#), see the [ControlChannelTrigger StreamWebSocket sample](#) [↗](#).

ControlChannelTrigger with HttpClient

Some special considerations apply when using [HttpClient](#) with [ControlChannelTrigger](#). There are some transport-specific usage patterns and best practices that should be followed when using a [HttpClient](#) with [ControlChannelTrigger](#). In addition, these considerations affect the way that requests to receive packets on the [HttpClient](#) are handled.

Note [HttpClient](#) using SSL is not currently supported using the network trigger feature and [ControlChannelTrigger](#). The following usage patterns and best practices should be followed when using [HttpClient](#) with [ControlChannelTrigger](#):

- The app may need to set various properties and headers on the [HttpClient](#) or [HttpClientHandler](#) object in the [System.Net.Http](#) namespace before sending the request to the specific URI.
- An app may need to make need to an initial request to test and setup the transport properly before creating the [HttpClient](#) transport to be used with [ControlChannelTrigger](#). Once the app determines that the transport can be properly setup, an [HttpClient](#) object can be configured as the transport object used with the [ControlChannelTrigger](#) object. This process is designed prevent some scenarios from breaking the connection established over the transport. Using SSL with a certificate, an app may require a dialog to be displayed for PIN entry or if there are multiple certificates to choose from. Proxy authentication and server authentication may be required. If the proxy or server authentication expires, the connection may be closed. One way an app can deal with these authentication

expiration issues is to set a timer. When an HTTP redirect is required, it is not guaranteed that the second connection can be established reliably. An initial test request will ensure that the app can use the most up-to-date redirected URL before using the [HttpClient](#) object as the transport with the [ControlChannelTrigger](#) object.

Unlike other network transports, the [HttpClient](#) object cannot be directly passed into the [UsingTransport](#) method of the [ControlChannelTrigger](#) object. Instead, an [HttpRequestMessage](#) object must be specially constructed for use with the [HttpClient](#) object and the [ControlChannelTrigger](#). The [HttpRequestMessage](#) object is created using the [RtcRequestFactory.Create](#) method. The [HttpRequestMessage](#) object that is created is then passed to [UsingTransport](#) method.

The following sample shows how to construct an [HttpRequestMessage](#) object for use with the [HttpClient](#) object and the [ControlChannelTrigger](#).

C#

```
using System;
using System.Net;
using System.Net.Http;
using System.Threading;
using System.Threading.Tasks;
using Windows.Networking.Sockets;

public HttpRequestMessage httpRequest;
public HttpClient httpClient;
public HttpRequestMessage httpRequest;
public ControlChannelTrigger channel;
public Uri serverUri;

private void SetupHttpRequestAndSendToHttpServer()
{
    try
    {
        // For HTTP based transports that use the RTC broker, whenever we
        // send next request, we will abort the earlier
        // outstanding http request and start new one.
        // For example in case when http server is taking longer to reply,
        // and keep alive trigger is fired in-between
        // then keep alive task will abort outstanding http request and
        // start a new request which should be finished
        // before next keep alive task is triggered.
        if (httpRequest != null)
        {
            httpRequest.Dispose();
        }

        httpRequest = RtcRequestFactory.Create(HttpMethod.Get, serverUri);
    }
}
```

```

        SendHttpRequest();
    }
    catch (Exception e)
    {
        Diag.DebugPrint("Connect failed with: " + e.ToString());
        throw;
    }
}

```

Some special considerations affect the way that requests to send HTTP requests on the [HttpClient](#) to initiate receiving a response are handled. In particular, when using a [HttpClient](#) with the [ControlChannelTrigger](#), your app must use a Task for handling sends instead of the **await** model.

Using [HttpClient](#), there is no synchronization with the [IBackgroundTask.Run](#) method on the background task for the [ControlChannelTrigger](#) with the return of the receive completion callback. For this reason, the app can only use the blocking [HttpResponseMessage](#) technique in the **Run** method and wait until the whole response is received.

Using [HttpClient](#) with [ControlChannelTrigger](#) is noticeably different from the [StreamSocket](#), [MessageWebSocket](#) or [StreamWebSocket](#) transports. The [HttpClient](#) receive callback is delivered via a Task to the app since the [HttpClient](#) code. This means that the [ControlChannelTrigger](#) push notification task will fire as soon as the data or error is dispatched to the app. In the sample below, the code stores the responseTask returned by [HttpClient.SendAsync](#) method into global storage that the push notify task will pick up and process inline.

The following sample shows how to handle send requests on the [HttpClient](#) when used with [ControlChannelTrigger](#).

C#

```

using System;
using System.Net;
using System.Net.Http;
using System.Threading;
using System.Threading.Tasks;
using Windows.Networking.Sockets;

private void SendHttpRequest()
{
    if (httpRequest == null)
    {
        throw new Exception("HttpRequest object is null");
    }

    // Tie the transport method to the controlchanneltrigger object to push

```

```

enable it.
    // Note that if the transport' s TCP connection is broken at a later
point of time,
    // the controlchanneltrigger object can be reused to plugin a new
transport by
    // calling UsingTransport API again.
    channel.UsingTransport(httpRequest);

    // Call the SendAsync function to kick start the TCP connection
establishment
    // process for this http request.
    Task<HttpResponseBody> httpResponseBodyTask =
httpClient.SendAsync(httpRequest);

    // Call WaitForPushEnabled API to make sure the TCP connection has been
established,
    // which will mean that the OS will have allocated any hardware slot for
this TCP connection.
    ControlChannelTriggerStatus status = channel.WaitForPushEnabled();
    Diag.DebugPrint("WaitForPushEnabled() completed with status: " +
status);
    if (status != ControlChannelTriggerStatus.HardwareSlotAllocated
        && status != ControlChannelTriggerStatus.SoftwareSlotAllocated)
    {
        throw new Exception("Hardware/Software slot not allocated");
    }

    // The HttpClient receive callback is delivered via a Task to the app.
    // The notification task will fire as soon as the data or error is
dispatched
    // Enqueue the responseBodyTask returned by httpClient.sendAsync
    // into a queue that the push notify task will pick up and process
inline.
    AppContext.messageQueue.Enqueue(httpResponseBodyTask);
}

```

The following sample shows how to read responses received on the [HttpClient](#) when used with [ControlChannelTrigger](#).

C#

```

using System.Net;
using System.Net.Http;
using System.Threading;
using System.Threading.Tasks;

public string ReadResponse(Task<HttpResponseBody> httpResponseBodyTask)
{
    string message = null;
    try
    {
        if (httpResponseBodyTask.IsCanceled || httpResponseBodyTask.IsFaulted)
        {

```



```

        Diag.DebugPrint("Task is cancelled or has failed");
        return message;
    }
    // We' ll wait until we got the whole response.
    // This is the only supported scenario for HttpClient for
    ControlChannelTrigger.
    HttpResponseMessage httpResponse = httpResponseTask.Result;
    if (httpResponse == null || httpResponse.Content == null)
    {
        Diag.DebugPrint("Cannot read from httpresponse, as either
        httpResponse or its content is null. try to reset connection.");
    }
    else
    {
        // This is likely being processed in the context of a background
        task and so
        // synchronously read the Content' s results inline so that the
        Toast can be shown.
        // before we exit the Run method.
        message = httpResponse.Content.ReadAsStringAsync().Result;
    }
}
catch (Exception exp)
{
    Diag.DebugPrint("Failed to read from httpresponse with error: " +
    exp.ToString());
}
return message;
}

```

For more information on using [HttpClient](#) with [ControlChannelTrigger](#), see the [ControlChannelTrigger HttpClient sample](#) [↗](#).

ControlChannelTrigger with IXMLHttpRequest2

Some special considerations apply when using [IXMLHttpRequest2](#) with [ControlChannelTrigger](#). There are some transport-specific usage patterns and best practices that should be followed when using an [IXMLHttpRequest2](#) with [ControlChannelTrigger](#). Using [ControlChannelTrigger](#) does not affect the way that requests to send or receive HTTP requests on the [IXMLHttpRequest2](#) are handled.

Usage patterns and best practices when using [IXMLHttpRequest2](#) with [ControlChannelTrigger](#)

- An [IXMLHttpRequest2](#) object when used as the transport has a lifetime of only one request/response. When used with the [ControlChannelTrigger](#) object, it is convenient to create and set up the [ControlChannelTrigger](#) object once and then call the [UsingTransport](#) method repeatedly, each time associating a new

IXMLHttpRequest2 object. An app should delete the previous

IXMLHttpRequest2 object before supplying a new **IXMLHttpRequest2** object to ensure that the app does not exceed the allocated resource limits.

- The app may need to call the [SetProperty](#) and [SetRequestHeader](#) methods to set up the HTTP transport before calling [Send](#) method.
- An app may need to make need to an initial [Send](#) request to test and setup the transport properly before creating the transport to be used with [ControlChannelTrigger](#). Once the app determines that the transport is properly setup, the **IXMLHttpRequest2** object can be configured as the transport object used with the **ControlChannelTrigger**. This process is designed prevent some scenarios from breaking the connection established over the transport. Using SSL with a certificate, an app may require a dialog to be displayed for PIN entry or if there are multiple certificates to choose from. Proxy authentication and server authentication may be required. If the proxy or server authentication expires, the connection may be closed. One way an app can deal with these authentication expiration issues is to set a timer. When an HTTP redirect is required, it is not guaranteed that the second connection can be established reliably. An initial test request will ensure that the app can use the most up-to-date redirected URL before using the **IXMLHttpRequest2** object as the transport with the **ControlChannelTrigger** object.

For more information on using [IXMLHttpRequest2](#) with [ControlChannelTrigger](#), see the [ControlChannelTrigger with IXMLHttpRequest2 sample](#) [↗](#).

Important APIs

- [SocketActivityTrigger](#)
- [ControlChannelTrigger](#)

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) [↗](#) | [Get help at Microsoft Q&A](#)

Sockets

Article • 10/20/2022

Sockets are a low-level data transfer technology on top of which many networking protocols are implemented. UWP offers TCP and UDP socket classes for client-server or peer-to-peer applications, whether connections are long-lived or an established connection is not required.

This topic focuses on how to use the Universal Windows Platform (UWP) socket classes that are in the [Windows.Networking.Sockets](#) namespace. But you can also use [Windows Sockets 2 \(Winsock\)](#) in a UWP app.

ⓘ Note

As a consequence of [network isolation](#), Windows disallows establishing a socket connection (Sockets or WinSock) between two UWP apps running on the same machine; whether that's via the local loopback address (127.0.0.0), or by explicitly specifying the local IP address. For details about mechanisms by which UWP apps can communicate with one another, see [App-to-app communication](#).

Build a basic TCP socket client and server

A TCP (Transmission Control Protocol) socket provides low-level network data transfers in either direction for connections that are long-lived. TCP sockets are the underlying feature used by most of the network protocols used on the Internet. To demonstrate basic TCP operations, the example code below shows a [StreamSocket](#) and a [StreamSocketListener](#) sending and receiving data over TCP to form an echo client and server.

To begin with as few moving parts as possible—and to sidestep network isolation issues for the present—create a new project, and put both the client and the server code below into the same project.

You'll need to [declare an app capability](#) in your project. Open your app package manifest source file (the `Package.appxmanifest` file) and, on the Capabilities tab, check **Private Networks (Client & Server)**. This is how that looks in the `Package.appxmanifest` markup.

XML

```
<Capability Name="privateNetworkClientServer" />
```

Instead of `privateNetworkClientServer`, you can declare `internetClientServer` if you're connecting over the internet. Both `StreamSocket` and `StreamSocketListener` need one or other of these app capabilities to be declared.

An echo client and server, using TCP sockets

Construct a `StreamSocketListener` and begin listening for incoming TCP connections. The `StreamSocketListener.ConnectionReceived` event is raised each time a client establishes a connection with the `StreamSocketListener`.

Also construct a `StreamSocket`, establish a connection to the server, send a request, and receive a response.

Create a new **Page** named `StreamSocketAndListenerPage`. Put the XAML markup in `StreamSocketAndListenerPage.xaml`, and then put the imperative code inside the `StreamSocketAndListenerPage` class.

XAML

```
<Grid Background="{ThemeResource ApplicationPageBackgroundThemeBrush}">
    <Grid.RowDefinitions>
        <RowDefinition Height="Auto"/>
        <RowDefinition Height="*/"/>
    </Grid.RowDefinitions>

    <StackPanel>
        <TextBlock Margin="9.6,0" Style="{StaticResource TitleTextBlockStyle}" Text="TCP socket example"/>
        <TextBlock Margin="7.2,0,0,0" Style="{StaticResource HeaderTextBlockStyle}" Text="StreamSocket & StreamSocketListener"/>
    </StackPanel>

    <Grid Grid.Row="1">
        <Grid.RowDefinitions>
            <RowDefinition/>
            <RowDefinition/>
        </Grid.RowDefinitions>
        <Grid.ColumnDefinitions>
            <ColumnDefinition Width="*/"/>
            <ColumnDefinition Width="*/"/>
        </Grid.ColumnDefinitions>
        <TextBlock Margin="9.6" Style="{StaticResource SubtitleTextBlockStyle}" Text="client"/>
        <ListBox x:Name="clientListBox" Grid.Row="1" Margin="9.6"/>
        <TextBlock Grid.Column="1" Margin="9.6" Style="{StaticResource SubtitleTextBlockStyle}" Text="server"/>
    </Grid>
</Grid>
```

```

        <ListBox x:Name="serverListBox" Grid.Column="1" Grid.Row="1"
Margin="9.6"/>
    </Grid>
</Grid>

```

C#

```

// Every protocol typically has a standard port number. For example, HTTP is
// typically 80, FTP is 20 and 21, etc.
// For this example, we'll choose an arbitrary port number.
static string PortNumber = "1337";

protected override void OnNavigatedTo(NavigationEventArgs e)
{
    this.StartServer();
    this.StartClient();
}

private async void StartServer()
{
    try
    {
        var streamSocketListener = new
Windows.Networking.Sockets.StreamSocketListener();

        // The ConnectionReceived event is raised when connections are
        // received.
        streamSocketListener.ConnectionReceived +=
this.StreamSocketListener_ConnectionReceived;

        // Start listening for incoming TCP connections on the specified
        // port. You can specify any port that's not currently in use.
        await
streamSocketListener.BindServiceNameAsync(StreamSocketAndListenerPage.PortNu
mber);

        this.serverListBox.Items.Add("server is listening...");
    }
    catch (Exception ex)
    {
        Windows.Networking.Sockets.SocketErrorStatus webErrorStatus =
Windows.Networking.Sockets.SocketError.GetStatus(ex.GetBaseException().HResu
lt);
        this.serverListBox.Items.Add(webErrorStatus.ToString() != "Unknown"
? webErrorStatus.ToString() : ex.Message);
    }
}

private async void
StreamSocketListener_ConnectionReceived(Windows.Networking.Sockets.StreamSoc
ketListener sender,
Windows.Networking.Sockets.StreamSocketListenerConnectionReceivedEventArgs
args)

```

```

{
    string request;
    using (var streamReader = new
StreamReader(args.Socket.InputStream.AsStreamForRead()))
    {
        request = await streamReader.ReadLineAsync();
    }

    await this.Dispatcher.RunAsync(CoreDispatcherPriority.Normal, () =>
this.serverListBox.Items.Add(string.Format("server received the request: \"{0}\"", request)));

    // Echo the request back as the response.
    using (Stream outputStream =
args.Socket.OutputStream.AsStreamForWrite())
    {
        using (var streamWriter = new StreamWriter(outputStream))
        {
            await streamWriter.WriteLineAsync(request);
            await streamWriter.FlushAsync();
        }
    }

    await this.Dispatcher.RunAsync(CoreDispatcherPriority.Normal, () =>
this.serverListBox.Items.Add(string.Format("server sent back the response: \"{0}\"", request)));

    sender.Dispose();

    await this.Dispatcher.RunAsync(CoreDispatcherPriority.Normal, () =>
this.serverListBox.Items.Add("server closed its socket"));
}

private async void StartClient()
{
    try
    {
        // Create the StreamSocket and establish a connection to the echo
server.
        using (var streamSocket = new
Windows.Networking.Sockets.StreamSocket())
        {
            // The server hostname that we will be establishing a connection
to. In this example, the server and client are in the same process.
            var hostName = new Windows.Networking.HostName("localhost");

            this.clientListBox.Items.Add("client is trying to connect...");

            await streamSocket.ConnectAsync(hostName,
StreamSocketAndListenerPage.PortNumber);

            this.clientListBox.Items.Add("client connected");

            // Send a request to the echo server.
            string request = "Hello, World!";

```

```

        using (Stream outputStream =
streamSocket.OutputStream.AsStreamForWrite())
        {
            using (var streamWriter = new StreamWriter(outputStream))
            {
                await streamWriter.WriteLineAsync(request);
                await streamWriter.FlushAsync();
            }
        }

        this.clientListBox.Items.Add(string.Format("client sent the
request: \"{0}\"", request));

        // Read data from the echo server.
        string response;
        using (Stream inputStream =
streamSocket.InputStream.AsStreamForRead())
        {
            using (StreamReader streamReader = new
StreamReader(inputStream))
            {
                response = await streamReader.ReadLineAsync();
            }
        }

        this.clientListBox.Items.Add(string.Format("client received the
response: \"{0}\" ", response));
    }

    this.clientListBox.Items.Add("client closed its socket");
}
catch (Exception ex)
{
    Windows.Networking.Sockets.SocketErrorStatus webErrorStatus =
Windows.Networking.Sockets.SocketError.GetStatus(ex.GetBaseException()).HResu
lt);
    this.clientListBox.Items.Add(webErrorStatus.ToString() != "Unknown"
? webErrorStatus.ToString() : ex.Message);
}
}

```

C++/WinRT

```

#include <winrt/Windows.Foundation.h>
#include <winrt/Windows.Networking.Sockets.h>
#include <winrt/Windows.Storage.Streams.h>
#include <winrt/Windows.UI.Core.h>
#include <winrt/Windows.UI.Xaml.Navigation.h>
#include <sstream>

using namespace winrt;
using namespace Windows::Foundation;
using namespace Windows::Storage::Streams;

```

```

using namespace Windows::UI::Core;
using namespace Windows::UI::Xaml::Navigation;
...
private:
    Windows::Networking::Sockets::StreamSocketListener
m_streamSocketListener;
    Windows::Networking::Sockets::StreamSocket m_streamSocket;

public:
    void OnNavigatedTo(NavigationEventArgs const& /* e */)
    {
        StartServer();
        StartClient();
    }

private:
    IAsyncAction StartServer()
    {
        try
        {
            // The ConnectionReceived event is raised when connections are
received.
            m_streamSocketListener.ConnectionReceived({ this,
&StreamSocketAndListenerPage::OnConnectionReceived });

            // Start listening for incoming TCP connections on the specified
port. You can specify any port that's not currently in use.
            // Every protocol typically has a standard port number. For
example, HTTP is typically 80, FTP is 20 and 21, etc.
            // For this example, we'll choose an arbitrary port number.
            co_await m_streamSocketListener.BindServiceNameAsync(L"1337");
            serverListBox().Items().Append(winrt::box_value(L"server is
listening..."));
        }
        catch (winrt::hresult_error const& ex)
        {
            Windows::Networking::Sockets::SocketErrorStatus webErrorStatus{
Windows::Networking::Sockets::SocketError::GetStatus(ex.to_abi()) };
            serverListBox().Items().Append(webErrorStatus !=
Windows::Networking::Sockets::SocketErrorStatus::Unknown ?
winrt::box_value(winrt::to_hstring((int32_t)webErrorStatus)) :
winrt::box_value(winrt::to_hstring(ex.to_abi())));
        }
    }

    IAsyncAction
OnConnectionReceived(Windows::Networking::Sockets::StreamSocketListener /*
sender */,
Windows::Networking::Sockets::StreamSocketListenerConnectionReceivedEventArg
s args)
    {
        try
        {
            auto socket{ args.Socket() }; // Keep the socket referenced, and
alive.

```



```

        DataReader dataReader{ socket.InputStream() };

        unsigned int bytesLoaded = co_await
dataReader.LoadAsync(sizeof(unsigned int));

        unsigned int stringLength = dataReader.ReadUInt32();
        bytesLoaded = co_await dataReader.LoadAsync(stringLength);
        winrt::hstring request = dataReader.ReadString(bytesLoaded);

serverListBox().Dispatcher().RunAsync(CoreDispatcherPriority::Normal, [=]()
{
    std::wstringstream wstringstream;
    wstringstream << L"server received the request: \"" <<
request.c_str() << L"\"";

serverListBox().Items().Append(winrt::box_value(wstringstream.str().c_str())
);

    });

    // Echo the request back as the response.
    DataWriter dataWriter{ socket.OutputStream() };
    dataWriter.WriteUInt32(request.size());
    dataWriter.WriteString(request);
    co_await dataWriter.StoreAsync();
    dataWriter.DetachStream();

serverListBox().Dispatcher().RunAsync(CoreDispatcherPriority::Normal, [=]()
{
    std::wstringstream wstringstream;
    wstringstream << L"server sent back the response: \"" <<
request.c_str() << L"\"";

serverListBox().Items().Append(winrt::box_value(wstringstream.str().c_str())
);

    });

    m_streamSocketListener = nullptr;

serverListBox().Dispatcher().RunAsync(CoreDispatcherPriority::Normal, [=]()
{
    serverListBox().Items().Append(winrt::box_value(L"server
closed its socket"));
    });
}
catch (winrt::hresult_error const& ex)
{
    Windows::Networking::Sockets::SocketErrorStatus webErrorStatus{
Windows::Networking::Sockets::SocketError::GetStatus(ex.to_abi()) };

serverListBox().Dispatcher().RunAsync(CoreDispatcherPriority::Normal, [=]()
{
    serverListBox().Items().Append(webErrorStatus !=

```

```

Windows::Networking::Sockets::SocketErrorStatus::Unknown ?
winrt::box_value(winrt::to_hstring((int32_t)webErrorStatus)) :
winrt::box_value(winrt::to_hstring(ex.to_abi())));
    });
}
}

IAsyncAction StartClient()
{
    try
    {
        // Establish a connection to the echo server.

        // The server hostname that we will be establishing a connection
        to. In this example, the server and client are in the same process.
        Windows::Networking::HostName hostName{ L"localhost" };

        clientListBox().Items().Append(winrt::box_value(L"client is
        trying to connect..."));

        co_await m_streamSocket.ConnectAsync(hostName, L"1337");
        clientListBox().Items().Append(winrt::box_value(L"client
        connected"));

        // Send a request to the echo server.
        DataWriter dataWriter{ m_streamSocket.OutputStream() };
        winrt::hstring request{ L"Hello, World!" };
        dataWriter.WriteUInt32(request.size());
        dataWriter.WriteString(request);

        co_await dataWriter.StoreAsync();

        std::wstringstream wstringstream;
        wstringstream << L"client sent the request: \"" <<
        request.c_str() << L"\"";

        clientListBox().Items().Append(winrt::box_value(wstringstream.str().c_str())
        );

        co_await dataWriter.FlushAsync();
        dataWriter.DetachStream();

        // Read data from the echo server.
        DataReader dataReader{ m_streamSocket.InputStream() };
        unsigned int bytesLoaded = co_await
        dataReader.LoadAsync(sizeof(unsigned int));
        unsigned int stringLength = dataReader.ReadUInt32();
        bytesLoaded = co_await dataReader.LoadAsync(stringLength);
        winrt::hstring response{ dataReader.ReadString(bytesLoaded) };

        wstringstream.str(L "");
        wstringstream << L"client received the response: \"" <<
        response.c_str() << L"\"";

        clientListBox().Items().Append(winrt::box_value(wstringstream.str().c_str())

```

```

);

        m_streamSocket = nullptr;

        clientListBox().Items().Append(winrt::box_value(L"client closed
its socket"));
    }
    catch (winrt::hresult_error const& ex)
    {
        Windows::Networking::Sockets::SocketErrorStatus webErrorStatus{
Windows::Networking::Sockets::SocketError::GetStatus(ex.to_abi()) };
        serverListBox().Items().Append(webErrorStatus !=
Windows::Networking::Sockets::SocketErrorStatus::Unknown ?
winrt::box_value(winrt::to_hstring((int32_t)webErrorStatus)) :
winrt::box_value(winrt::to_hstring(ex.to_abi())));
    }
}

```

C++

```

#include <ppltasks.h>
#include <sstream>
...
using namespace Windows::Foundation;
using namespace Windows::Storage::Streams;
using namespace Windows::UI::Core;
using namespace Windows::UI::Xaml::Navigation;
...
private:
    Windows::Networking::Sockets::StreamSocketListener^
streamSocketListener;
    Windows::Networking::Sockets::StreamSocket^ streamSocket;

protected:
    virtual void OnNavigatedTo(NavigationEventArgs^ e) override
    {
        this->StartServer();
        this->StartClient();
    }

private:
    void StartServer()
    {
        try
        {
            this->streamSocketListener = ref new
Windows::Networking::Sockets::StreamSocketListener();

            // The ConnectionReceived event is raised when connections are
received.
            streamSocketListener->ConnectionReceived += ref new
TypedEventHandler<Windows::Networking::Sockets::StreamSocketListener^,
Windows::Networking::Sockets::StreamSocketListenerConnectionReceivedEventArgs>

```

```

s^>(this,
&StreamSocketAndListenerPage::StreamSocketListener_ConnectionReceived);

    // Start listening for incoming TCP connections on the specified
    port. You can specify any port that's not currently in use.
    // Every protocol typically has a standard port number. For
    example, HTTP is typically 80, FTP is 20 and 21, etc.
    // For this example, we'll choose an arbitrary port number.
    Concurrency::create_task(streamSocketListener-
>BindServiceNameAsync(L"1337")).then(
    [=]
    {
        this->serverListBox->Items->Append(L"server is
listening...");
    });
}
catch (Platform::Exception^ ex)
{
    Windows::Networking::Sockets::SocketErrorStatus webErrorStatus =
Windows::Networking::Sockets::SocketError::GetStatus(ex->HRESULT);
    this->serverListBox->Items->Append(webErrorStatus.ToString() !=
L"Unknown" ? webErrorStatus.ToString() : ex->Message);
}
}

void
StreamSocketListener_ConnectionReceived(Windows::Networking::Sockets::Stream
SocketListener^ sender,
Windows::Networking::Sockets::StreamSocketListenerConnectionReceivedEventArg
s^ args)
{
    try
    {
        auto dataReader = ref new DataReader(args->Socket->InputStream);

        Concurrency::create_task(dataReader->LoadAsync(sizeof(unsigned
int))).then(
            [=](unsigned int bytesLoaded)
            {
                unsigned int stringLength = dataReader->ReadUInt32();
                Concurrency::create_task(dataReader-
>LoadAsync(stringLength)).then(
                    [=](unsigned int bytesLoaded)
                    {
                        Platform::String^ request = dataReader-
>ReadString(bytesLoaded);
                        this->Dispatcher-
>RunAsync(CoreDispatcherPriority::Normal, ref new DispatchedHandler(
                            [=]
                            {
                                std::wstringstream wstringstream;
                                wstringstream << L"server received the request: \""
<< request->Data() << L"\\"";
                                this->serverListBox->Items->Append(ref new
Platform::String(wstringstream.str().c_str()));

```

```

    }));

    // Echo the request back as the response.
    auto dataWriter = ref new DataWriter(args->Socket->OutputStream);

    dataWriter->WriteUInt32(request->Length());
    dataWriter->WriteString(request);
    Concurrency::create_task(dataWriter->StoreAsync()).then(
        [=](unsigned int)
        {
            dataWriter->DetachStream();

            this->Dispatcher->RunAsync(CoreDispatcherPriority::Normal, ref new DispatchedHandler(
                [=]()
                {
                    std::wstringstream wstringstream;
                    wstringstream << L"server sent back the
response: \"" << request->Data() << L"\"";
                    this->serverListBox->Items->Append(ref new
Platform::String(wstringstream.str().c_str()));
                }));

            delete this->streamSocketListener;
            this->streamSocketListener = nullptr;

            this->Dispatcher->RunAsync(CoreDispatcherPriority::Normal, ref new DispatchedHandler([=]()
            {this->serverListBox->Items->Append(L"server closed its socket"); }));
        }));
    });
}
catch (Platform::Exception^ ex)
{
    Windows::Networking::Sockets::SocketErrorStatus webErrorStatus =
Windows::Networking::Sockets::SocketError::GetStatus(ex->HRESULT);
    this->Dispatcher->RunAsync(CoreDispatcherPriority::Normal, ref
new DispatchedHandler([=]() {this->serverListBox->Items->Append(webErrorStatus.ToString() != L"Unknown" ? webErrorStatus.ToString()
: ex->Message); }));
}

void StartClient()
{
    try
    {
        // Create the StreamSocket and establish a connection to the
echo server.
        this->streamSocket = ref new
Windows::Networking::Sockets::StreamSocket();

        // The server hostname that we will be establishing a connection
to. In this example, the server and client are in the same process.

```